ASSIGNMENT 3 OUTPUTS

TASK-1

```
    void display() const {
        cout << "Title: " << title << ", Copies: " << copies
             << ", Author: " << author << '\n';
    }
};

int main() {
    try {
        Book empty;
        Book donated("The Alchemist");
        Book partial("C++ Basics", 4);
        Book full("OOP in C++", 6, "Bjarne");
        empty.display();
        donated.display();
        partial.display();
        full.display();

        try {
            Book invalid("Invalid Book", -2);
        } catch (const invalid_argument& e) {
            cout << "Rejected: " << e.what() << '\n';
        }
    } catch (const exception& e) {
        cout << "Error: " << e.what() << '\n';
    }
    return 0;
}
```

input

```
Title: The Alchemist, Copies: 1, Author: Unknown
Title: C++ Basics, Copies: 4, Author: Unknown
Title: OOP in C++, Copies: 6, Author: Bjarne
Rejected: Copies cannot be negative

...Program finished with exit code 0
Press ENTER to exit console.
```

TASK-2

```
            medicines = fresh;
            n = other.n;
        }
        return *this;
    }

    ~Prescription() { delete[] medicines; }

    void set(int i, const string& m) { if (i >= 0 && i < n) medicines[i] = m; }
    string get(int i) const { return (i >= 0 && i < n) ? medicines[i] : ""; }
};

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\n';
    r = r; // self-assignment test
    return 0;
}
```

input

```
Copy constructor called
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS


...Program finished with exit code 0
Press ENTER to exit console.
```

ASK- 3

```
            for (int i = 0; i < rows; ++i) delete[] map[i];
            delete[] map;
        }
        cout << "Event closed. Booked seats: " << booked << '\n';
    }
};

int main() {
    Event local(2, 3);
    local.book(0, 1);
    local.book(1, 2);
    local.cancel(0, 1);
    cout << "Local event map:\n";
    local.display();

    {
        Event blockEvent(1, 2);
        blockEvent.book(0, 0);
    }

    Event* heapEvent = new Event(1, 2);
    heapEvent->book(0, 1);
    delete heapEvent;

    Event zero(0, 4);
    cout << "Zero-row event created safely\n";
    return 0;
}
```

input

```
Event created: 0 rows x 4 seats
Zero-row event created safely
Event closed. Booked seats: 0
Event closed. Booked seats: 1


...Program finished with exit code 0
Press ENTER to exit console.
```

TASK-4

main.cpp

```
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }

    friend ostream& operator<<(ostream& out, const Money& m) {
        long long value = m.paise;
        if (value < 0) {
            out << "-Rs " << (-value) / 100 << '.'
                << setw(2) << setfill('0') << (-value) % 100 << setfill(' ');
        } else {
            out << "Rs " << value / 100 << '.'
                << setw(2) << setfill('0') << value % 100 << setfill(' ');
        }
        return out;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';

    Money x(0, 75), y(0, 50);
    cout << "Paise carry: " << x + y << '\n';
    return 0;
}
```

input

```
Rs 200.00
true
Paise carry: Rs 1.25


...Program finished with exit code 0
Press ENTER to exit console.
```

TASK-5

```
    const string& operator[](int i) const {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw out_of_range("Playlist index out of range");
        return tracks[i];
    }
    int size() const { return static_cast<int>(tracks.size()); }
};

int main() {
    Time t(5);
    Time a = t++;
    Time b = ++t;
    cout << "a=" << a.value() << " b=" << b.value()
         << " t=" << t.value() << '\n';

    Playlist p{"Song One", "Song Two", "Song Three"};
    cout << "Playing: " << p[0] << '\n';
    p[1] = "Updated Song";
    cout << "Updated: " << p[1] << '\n';

    try { cout << p[3] << '\n'; }
    catch (const out_of_range& e) { cout << "Error: " << e.what() << '\n'; }

    Time end(3600), next(3601);
    cout << boolalpha << "end < next: " << (end < next) << '\n';
    return 0;
}
```

input

```
Playing: Song One
Updated: Updated Song
Error: Playlist index out of range
end < next: true


...Program finished with exit code 0
Press ENTER to exit console.
```

TASK-6

```
    Text operator+(const Text& other) const {
        char* joined = new char[strlen(data) + strlen(other.data) + 1];
        strcpy(joined, data);
        strcat(joined, other.data);
        Text result(joined);
        delete[] joined;
        return result;
    }

    bool operator==(const Text& other) const { return strcmp(data, other.data) == 0; }
    bool operator!=(const Text& other) const { return !(*this == other); }

    friend ostream& operator<<(ostream& out, const Text& t) {
        return out << t.data;
    }
};

int main() {
    Text a("Hello "), b("World"), c("");
    Text combined = a + b + c;
    cout << "Combined: " << combined << '\n';
    cout << boolalpha << "Equal: " << (combined == Text("Hello World")) << '\n';
    combined = combined; // self-assignment
    Text empty;
    cout << "Empty text: [" << empty << "]\n";
    return 0;
}
```

input

```
Combined: Hello World
Equal: true
Empty text: []


...Program finished with exit code 0
Press ENTER to exit console.
```

TASK-7

```
        out << z.real;
        if (z.imag >= 0) out << " + j" << z.imag;
        else out << " - j" << -z.imag;
        return out;
    }
};

int main() {
    Complex z1(4, 3), z2(2, -1);
    Complex series = z1 + z2;
    Complex parallel = (z1 * z2) / (z1 + z2);

    cout << "Z1 = " << z1 << '\n';
    cout << "Z2 = " << z2 << '\n';
    cout << "Series impedance = " << series << '\n';
    cout << "Series magnitude = " << series.magnitude() << '\n';
    cout << "Parallel impedance = " << parallel << '\n';
    cout << "Parallel magnitude = " << parallel.magnitude() << '\n';

    try {
        Complex invalid = z1 / Complex(0, 0);
        cout << invalid << '\n';
    } catch (const runtime_error& e) {
        cout << "Safe error: " << e.what() << '\n';
    }
    return 0;
}
```

input

```
Series magnitude = 6.32456
Parallel impedance = 1.75 - j0.25
Parallel magnitude = 1.76777
Safe error: Division by zero impedance


...Program finished with exit code 0
Press ENTER to exit console.
```